From Text To Attention

How Transformers See Language

Part 1

Connecting conceptual foundations to mechanical implementation.

Bridging the Knowledge Gap

Filling the gaps in our understanding of Neural Networks

What we know

- ✓ Neural networks process numbers
- ✓ Models learn by adjusting weights
- ✓ Attention focuses on relevant words

The missing pieces

- ? How does text transform into numbers?
- ? What does the model actually "see"?
- ? How does attention work mechanically?

The Fundamental Challenge

Neural networks only understand **NUMBERS**.

Human language is **TEXT**.

We need a translation layer that converts text into numbers while preserving semantic relationships.

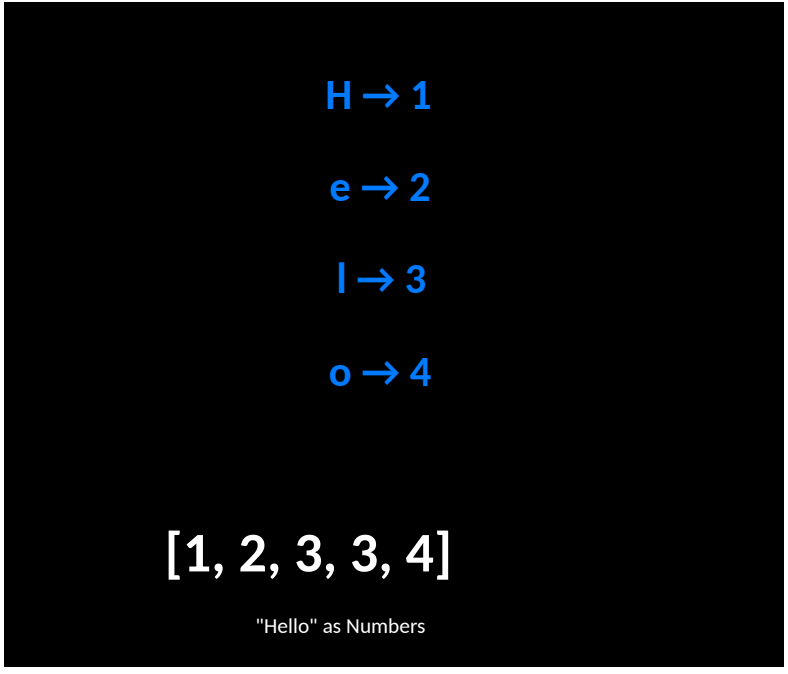
So… how?

The Character-Level Approach

Idea 1: One number per character

The most basic way to achieve numerical representation is to assign a **unique integer** to every individual character in the alphabet.

While simple, this approach operates at the smallest possible unit of language, ignoring words and concepts entirely.



Why Character-Level Models Struggle

The hidden cost of extreme granularity

The Step Problem

A word like **"understanding"** needs **13 separate steps** to process — the model must work many times harder to see one concept.

Manual Pattern Learning

The model must learn prefixes and roots from individual characters rather than receiving those units directly.

High Cognitive Load

It's like teaching reading by showing letters only — building grammar and meaning becomes much harder.

Extreme Inefficiency

Large amounts of data are spent learning basic vocabulary instead of higher-level logic.

The Word-Level Approach

Idea 2: Assigning one unique number to every whole word

The Concept

```
"The cat sat"
↓
[1, 2, 3]
```

/ **Concept Mapping:** Each word represents a single, discrete unit of meaning.

/ Much more intuitive than character-level processing.

The Challenges

! **The "Form" Problem:** Are "cat", "cats", and "cat's" three different concepts?

! **The Unknown:** How to handle typos like "teh" or rare technical terms?

! **Out of Vocabulary:** Any word not in the training set becomes a "hole" in understanding.

The Vocabulary Explosion

Scaling challenges of word-level models

170,000+

Common English Words

3 Billion

Parameters for Lookup

Computational Bottleneck

Parameter Bloat

Each unique word needs its own embedding row; huge vocabularies create large memory overhead.

The "Unknown" Problem

Fixed vocabularies fail on names, slang, and new terms, reducing robustness.

Multilingual Scaling

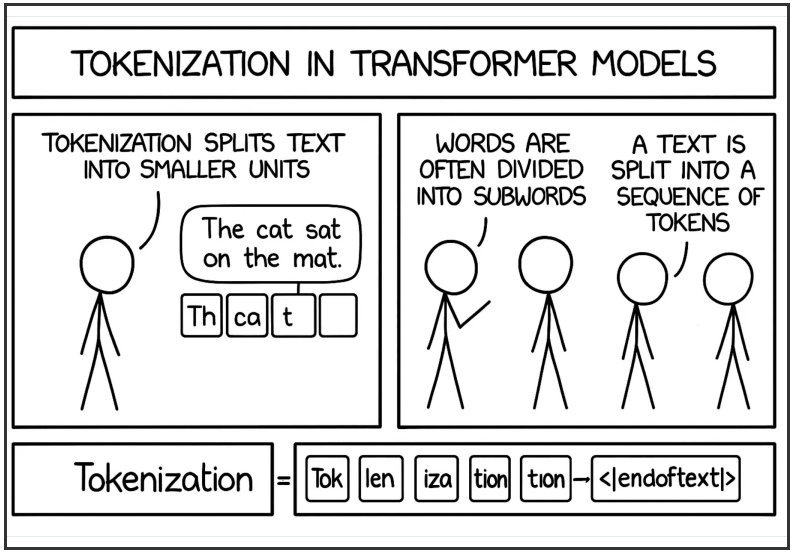
Adding languages multiplies vocabulary size, driving up compute and storage costs.

The Subword Solution

Finding the "Goldilocks" balance between characters and words

Strategic Splitting

- **The Middle Ground:** Bigger than characters (too granular) but smaller than whole words (too many).
- **Efficiency:** Common words stay whole; rare words break into recognizable, meaningful pieces.
- **Example:** "understanding" → ["under", "stand", "ing"]
- A vocabulary of 30k–100k tokens can represent almost any text in any language.



Byte Pair Encoding (BPE)

Algorithmic construction of modern tokenizers

1. Initialization

Begin with each character as an individual token.

2. Iterative Merging

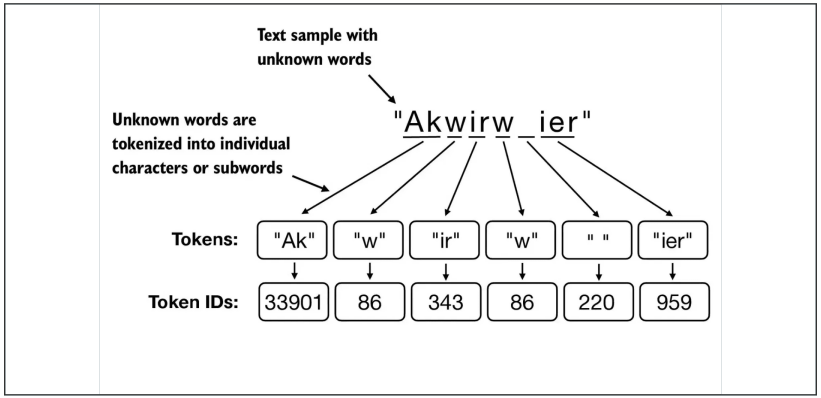
Merge the most frequent adjacent token pair into one.

3. Optimization

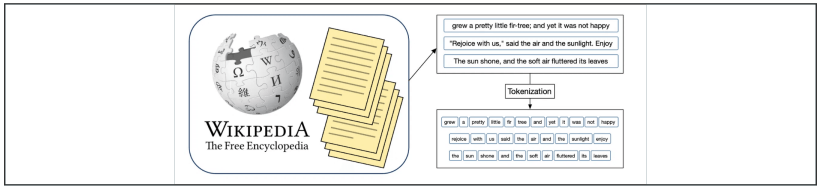
Repeat merges until reaching the target vocabulary size.

4. Self-Organizing

Frequent patterns become single tokens; rare ones stay fragmented.



Visualizing the Token Merging Process



Tokenization in Modern Practice

How GPT-style tokenizers process real-world text

Space Handling

```
"Hello world"
↓
["Hello", " world"]
```

Spaces are often attached to the beginning of the following word, not treated as separate tokens.

Contractions

```
"don't"
↓
["don", "'t"]
```

Common contractions are split into specific markers that the model has learned to associate with negation or possession.

Complex Words

```
"artificial intelligence"
↓
["art", "ificial", " intelligence"]
```

Long or rare words are broken into subword units that might not always align with linguistic syllables.

Impact: Understanding these splits is crucial for managing context limits and API costs.

Why Tokenization Matters

The practical implications of how models "see" text

Operational Impact

Context Limits

GPT-4's **128K limit** refers to tokens, not words. In English, this is roughly 1.3 tokens per word, but code can be much higher.

Financial Cost

API billing is calculated based on **token counts**, not character counts. Efficient prompting requires understanding token density.

Model Behavior

Logical Failures

Strange failures in **counting letters** or reversing words often stem from the model seeing subword units rather than individual characters.

Language Equity

Different languages require different numbers of tokens to express the same concept, affecting both **cost and performance** globally.

Summary: The Model's "Eyes"

Tokenization as the foundation of transformer processing

The Granularity Balance

Characters → Too Granular

Words → Too Many

Subwords → Just Right

The tokenizer determines the fundamental units the model uses for "thought."

Operational Impact

Poor tokenization makes the model work harder to understand context and can lead to strange failures in basic tasks.

Perception

If the model can't "see" a pattern in the tokens, it can't learn the underlying meaning of the text.

Next: Moving from arbitrary IDs to semantic meaning

From Arbitrary IDs to Meaning

Moving beyond simple integer labels

The Problem

```
"Hello world"
↓
[15496, 995]
```

These IDs are **arbitrary**. The number 15496 doesn't "know" it represents a greeting, and 995 doesn't "mean" anything about the Earth.

The Goal

We need representations that capture MEANING.

We want to transform these discrete, arbitrary integers into continuous, dense vectors where the numbers themselves encode semantic relationships.

In this new space, "Hello" and "Hi" should be mathematically similar, while "Hello" and "Banana" should be distant.

The Concept of Embeddings

Representing tokens as points in high-dimensional space

Spatial Logic

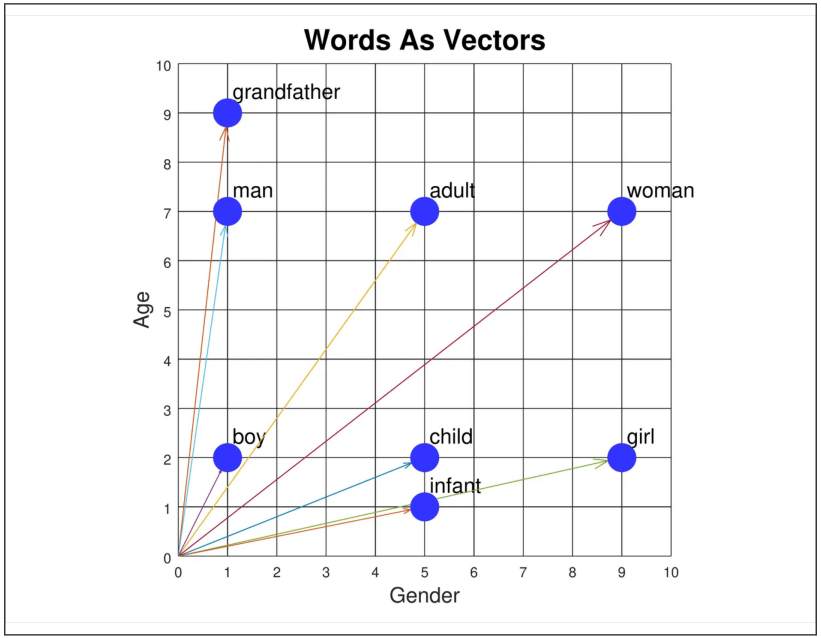
Tokens with similar meanings are placed **near each other** in the vector space. Proximity equals semantic similarity.

Vector Representation

Each token becomes a list of numbers (a vector) instead of a single ID, allowing for mathematical operations on meaning.

Multidimensionality

Modern models use **hundreds or thousands** of dimensions to capture the subtle nuances of human language.



Visualizing Semantic Space

How models organize concepts across dimensions

Clustering

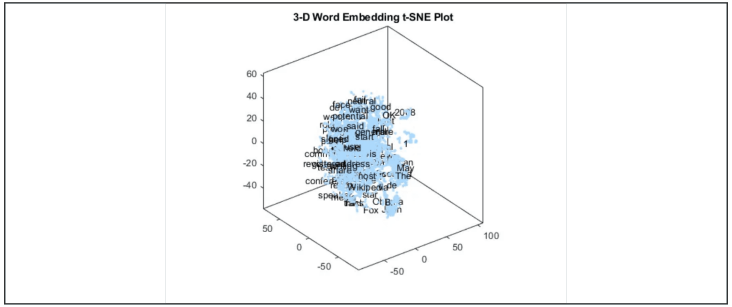
Related concepts (e.g., royalty, gender, animals) naturally group together in the high-dimensional vector space.

Emergent Properties

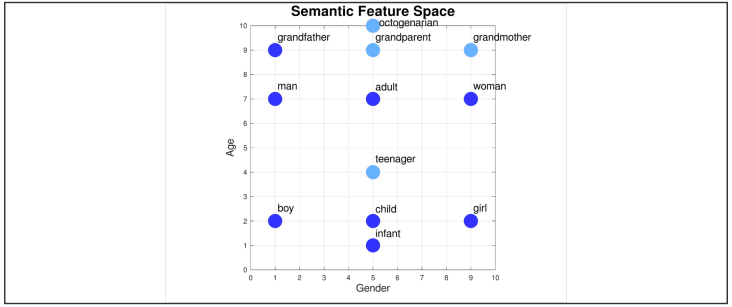
Dimensions like "gender" or "royalty" emerge automatically during training without manual labeling.

Semantic Navigation

The mathematical "distance" between points represents the semantic similarity of the tokens.



3D t-SNE Visualization of Word Clusters



2D Semantic Feature Space Mapping

Semantic Vector Arithmetic

Mathematical logic embedded in language

king - man + woman ≈ queen

Directional Meaning

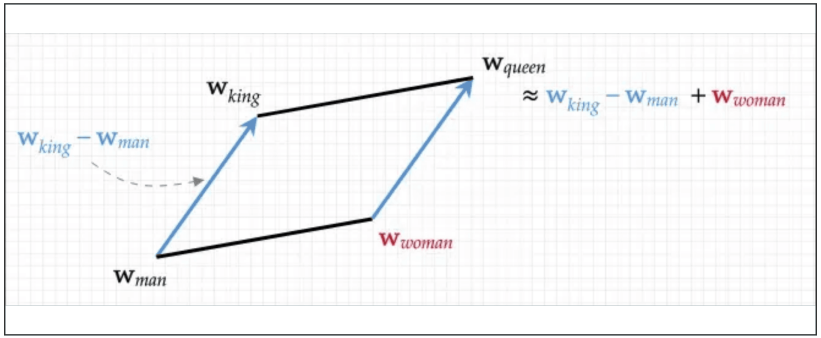
The man→woman vector mirrors king→queen.

Relational Encoding

Embeddings encode relations as consistent offsets.

Universal Patterns

These offsets recur across large vocabularies.



Visualizing Vector Logic in Semantic Space

Embeddings in the Pipeline

How tokens transform into dense vectors

The Lookup Table

```
ID 15496
↓
[0.12, -0.45, 0.88, ...]
```

Each token ID acts as an **index** to a specific row in the embedding matrix. This is a simple, fast lookup operation.

The result is a **768-dimensional vector** (for base models) that represents the token's core semantic profile.

Information Density

A single vector packs multiple layers of semantic information, capturing nuance that a single integer ID never could.

The "Raw" Starting Point

This vector is the **isolated representation** of the word. It contains the general meaning of the token before any context from the surrounding sentence is applied.

Summary: Embeddings

Converting discrete tokens into meaningful continuous space

Core Functions

- Transform arbitrary integer IDs into points in a high-dimensional semantic map.
- Ensure proximity in vector space equals similarity in meaning.
- Pack multiple layers of semantic nuance into a single dense vector.

Foundational Layer

Every transformer starts by looking up these pre-trained meanings. It is the bedrock of all subsequent processing.

The Limitation

Embeddings are "bag-of-words" by default. They represent isolated meaning but ignore the order of words.

Next: Solving the problem of sequence and word order

The Problem of Sequence

Attention is "bag-of-words" by default

Order Neutrality

"The dog bit the man"
vs.
"The man bit the dog"

Without extra information, attention treats these two sentences **identically**. The mathematical operations don't care about position.

This is known as **permutation invariance**—the model sees a set of words, not a sequence.

The Loss of Logic

In human language, **position** is often the primary driver of meaning. Order determines who did what to whom.

The Requirement

We must find a way to inject "where" a word is into "what" a word is. We need to make the model aware of the **dimension of time** in language.

Positional Encoding

Adding a sense of time and order to the model

Injecting Sequence

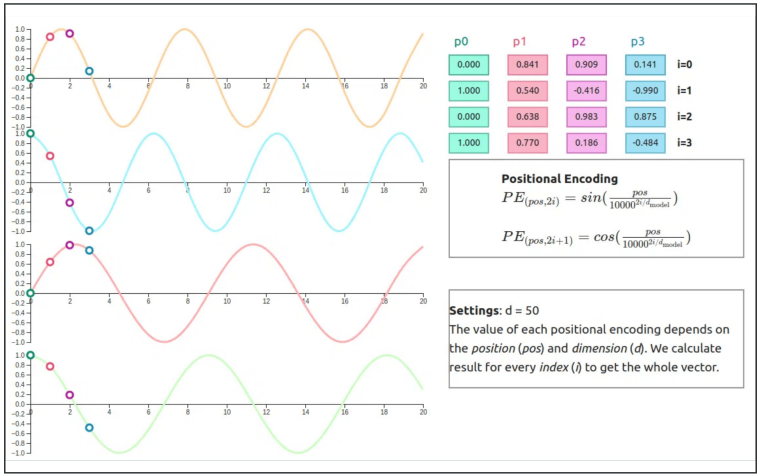
Transformers process tokens in parallel, so we add tags to show **where** each token sits in the sequence.

Vector Addition

A small positional vector is added to each embedding, encoding both **meaning** and **position**.

Spatial Awareness

This helps the model tell apart different word orders (e.g. who did what) using position tags.



How Positional Patterns Work

Using mathematical waves to encode location

Periodic Functions

We use **sine and cosine** waves of varying frequencies. Each dimension of the embedding follows a different wave.

Unique Fingerprint

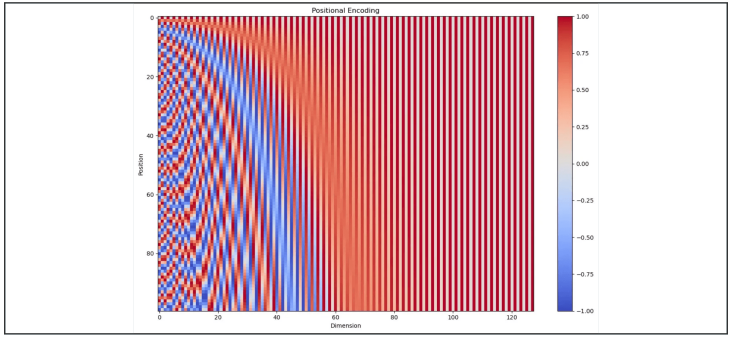
Every position in the sequence (0, 1, 2...) gets a unique combination of values, creating a "fingerprint" for that location.

Relative Distance

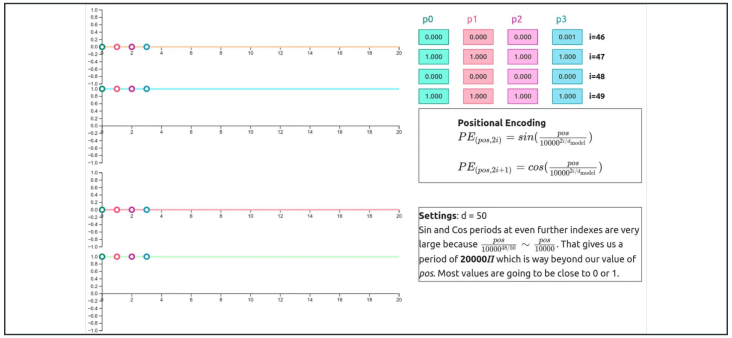
The model can calculate the **relative distance** between words because the waves are mathematically related.

Generalization

This method allows the model to handle sequences longer than those it saw during training.



Sine and Cosine Wave Patterns



Unique Encoding Matrix for Positions

Why Sine Waves?

The logic behind the original transformer's choice

Mathematical Advantages

Relative Position

For any fixed offset **k**, the position **p+k** can be represented as a linear function of position **p**. This allows the model to learn to attend by relative position easily.

Generalization

Since the waves are continuous, the model can extrapolate and handle **longer sequences** than those it saw during training.

Efficiency & Logic

No Learned Parameters

Unlike other methods, sinusoidal encoding doesn't require extra parameters to learn. The patterns are **fixed and pre-calculated**.

Unique Multi-Scale Patterns

By using multiple frequencies, the model creates a **unique fingerprint** for every single position in a very high-dimensional space.

The Pipeline So Far

From raw text to context-ready vectors

Step 1

Tokenization

Breaking raw text into discrete subword units (IDs).

Step 2

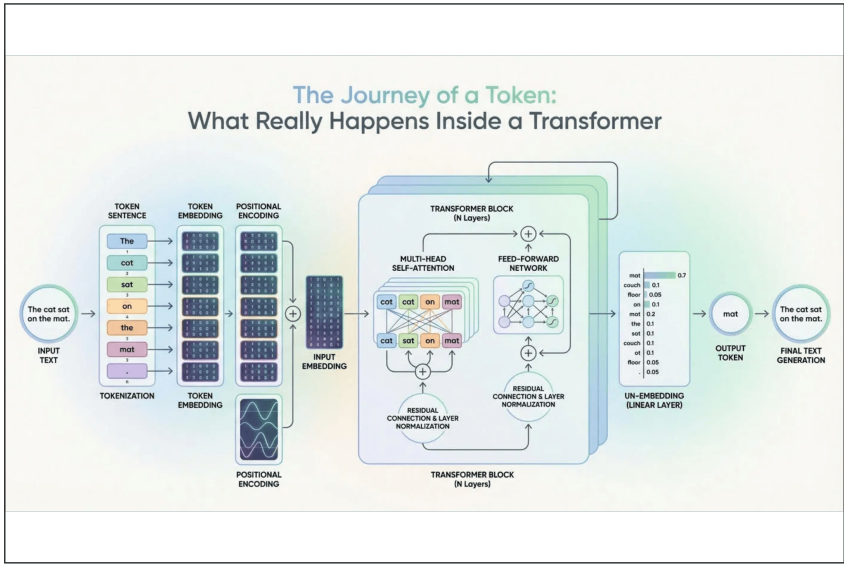
Embedding

Converting arbitrary IDs into dense semantic vectors.

Step 3

Positional Encoding

Injecting sequence awareness via mathematical wave patterns.



The Core of the Transformer

Introducing the Self-Attention mechanism

The Engine of Context

Self-Attention

The defining feature that allows the model to process information **globally** rather than sequentially. It is the heart of the "Attention is All You Need" philosophy.

Mutual Influence

Every token in a sequence "attends" to every other token simultaneously, creating a rich, multi-layered map of relationships.

Why It Matters

Contextualization

Words don't have a single fixed meaning; their meaning is **refined and updated** based on the words surrounding them in the specific sequence.

Interconnectivity

Self-attention builds a dense web of connections, allowing the model to resolve ambiguities and understand complex logical dependencies.

The Intuition of Attention

Focusing on what matters for understanding

Selective Focus

Filtering Noise

Attention allows the model to **ignore irrelevant words** and amplify the signal from the most related tokens in the sequence.

Dynamic Weighting

The model assigns a unique "importance score" to every word pair, determining exactly how much they should influence each other.

Reference Resolution

Solving Ambiguity

In the sentence "The bank was closed because **it** was a holiday," attention helps the model know that "it" refers to the "bank."

Semantic Context

The meaning of a word is defined by its relationships. Attention is the tool the model uses to **map those relationships**.

The Three Roles of Every Word

Queries, Keys, and Values (Q, K, V)

Query (Q)

"What am I looking for?" The search criteria used by a token to find relevant information in the sequence.

Key (K)

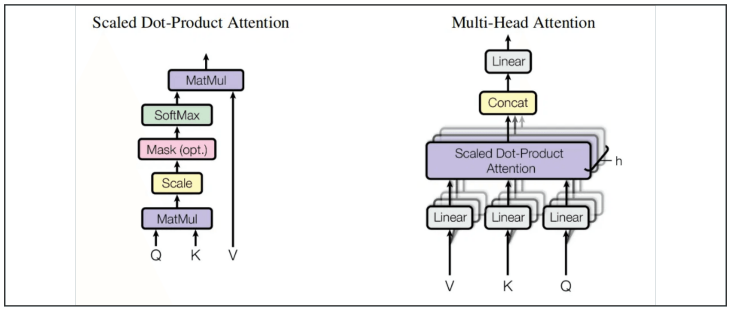
"What do I contain?" The label or index that other tokens use to determine if this token is relevant to their search.

Value (V)

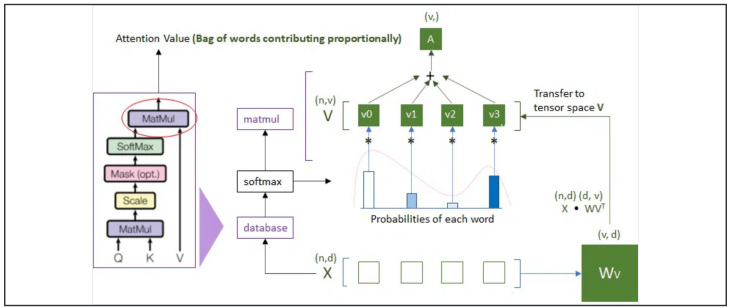
"What information do I offer?" The actual semantic content that is passed through if the Query and Key match.

Interaction

The **similarity score** between Q and K determines the "attention weight" given to the corresponding Value.



The Q, K, V Information Retrieval Model



Calculating Attention via Q, K, and V

The Database Analogy

Understanding attention as a "soft" lookup

Standard vs. Soft

Hard Lookup

A traditional database finds an **exact match** for a key and returns a single value. It is binary: match or no match.

Soft Lookup (Attention)

Attention finds **partial matches** across all keys. It returns a mixture of values based on how well each key matches the query.

The Mathematics

Weighted Sum

The final representation of a word is a **weighted combination** of all other words in the sequence.

Softmax Function

Softmax converts raw similarity scores into **probabilities** that sum to 1, determining the "mix" of values.

Computing Q, K, and V

The mathematical transformation of embeddings

The Transformation

Projection Matrices

A single word embedding is multiplied by three separate **learned matrices** (W_Q, W_K, W_V) to create its Query, Key, and Value vectors.
Input em

Learned Behavior

During training, the model learns exactly how to project embeddings to maximize the usefulness of the attention mechanism.

Three Perspectives

The Searcher (Q)

The version of the word used to **find context** in other words.

The Content (V)

The version of the word that **provides information** once a match is found.

The Label (K)

The version used by **other words** to find this one.

The Attention Formula

Breaking down the famous equation

The Equation

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k})\, V$$

This single formula defines the core logic of modern AI, allowing for the **dynamic exchange** of information across a sequence.

The Components

(3,4) (4,3) = (3,3)

Dot Product (QK^T)

Measures the **similarity** between every Query and every Key in the sequence.

Scaling ($\sqrt{d_k}$)

Prevents the dot products from becoming too large, ensuring stable gradients during training.

Softmax

Converts raw similarity scores into **attention weights** (probabilities) that sum to 1.

Weighted Sum (V)

Applies the weights to the Values to produce the final **context-aware** representation.

Visualizing the Attention Matrix

Mapping how words connect to each other

Mapping Connections

The attention matrix is a **square grid** where every word in the sequence is compared to every other word.

Heatmap Logic

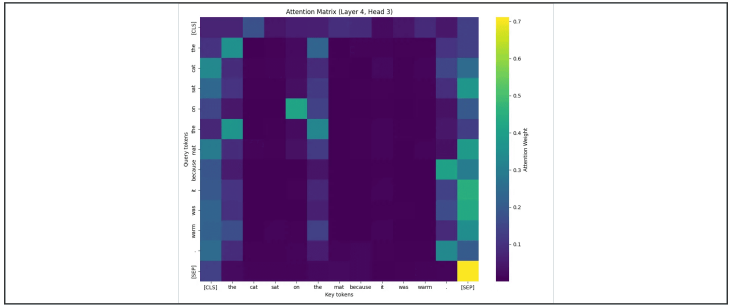
Each cell represents an attention weight. Brighter cells indicate a **stronger relationship** between the tokens.

Self-Focus

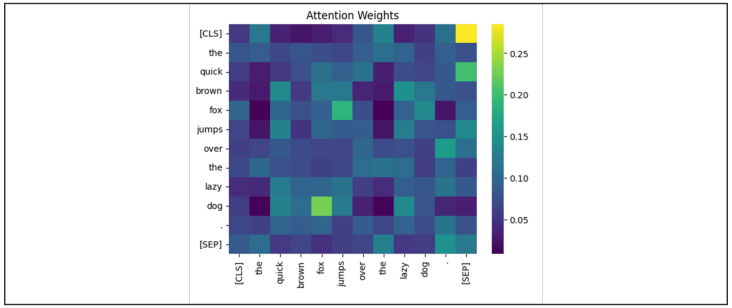
Words often attend strongly to themselves, preserving their own identity while incorporating context.

Context-Focus

The model learns to attend to logically related tokens, such as **pronouns to their antecedents** or verbs to their subjects.



The All-to-All Attention Map



Visualizing Inter-Word Relationships

Context-Aware Meaning

How attention solves ambiguity in language

The Problem

"I went to the bank..."
vs.
"The bank was closed..."

In isolation, the word "bank" is **ambiguous**. It could mean a financial institution or the side of a river.

Static embeddings provide a "blended" average meaning, which is often insufficient for deep understanding.

The Solution

Attention allows the model to **update the representation** of "bank" based on the surrounding tokens.

Disambiguation

By attending to words like "money," "loan," or "river," the model pulls in specific features that **resolve the ambiguity**, creating a context-aware representation.

A CONCRETE WALKTHROUGH

Tracking "it" through the attention mechanism

THE EXAMPLE

> "The animal didn't cross the street because it was too tired."

To understand this sentence, the model must resolve the ambiguity of the word **"it."** Does it refer to the animal or the street?

ATTENTION STEPS

1. THE QUERY

The word "it" sends out a Query: **"What noun am I referring to?"**

2. THE MATCH

The word "animal" has a Key that matches the Query. "Street" also matches, but less strongly in this context.

3. THE UPDATE

The representation of "it" is **updated** by pulling in semantic features from "animal."

4. THE RESULT

The model now "knows" that "it" refers to the animal, enabling correct logical processing.

The Generation Flow Looks like:

Generating text token by token:

Step 1: "The" → compute Q, K, V → attention → next token "cat"
Step 2: "The cat" → compute Q, K, V → attention → next token "sat"
Step 3: "The cat sat" → compute Q, K, V → attention → next token "on"
And so on ...

Each step, we run attention over the ENTIRE sequence. What's actually being computed?

What Does the New Token Need?

When generating token 4 ("on"), what do we compute?

The NEW token needs to attend to ALL previous tokens.

New token's Q: "What am I looking for?"
→ Needs fresh computation (depends on new token)

Previous tokens' K: "What do they contain?"
→ "The" K, "cat" K, "sat" K
→ These DON'T CHANGE — same tokens, same keys

Previous tokens' V: "What do they contribute?"
→ "The" V, "cat" V, "sat" V
→ These DON'T CHANGE either

Only the NEW token's Q changes each step!

The Waste

For Generating "cat"
  Compute: Q_the, K_the, V_the ← needed
        Q_cat, K_cat, V_cat ← needed (new token)

For Generating "sat"
  Compute: Q_the, K_the, V_the ← SAME AS BEFORE!
        Q_cat, K_cat, V_cat ← SAME AS BEFORE!
        Q_sat, K_sat, V_sat ← needed (new token)

We only need Q for the NEW token.
But K and V for OLD tokens? Already computed. Wasted work.

KV Cache

Instead of recomputing K and V for all tokens…
STORE them after first computation.

Step 1: "The" → compute K_the, V_the → CACHE them
Step 2: "cat" → compute K_cat, V_cat → CACHE them
use cached K_the, V_the
Step 3: "sat" → compute K_sat, V_sat → CACHE them
use cached K_the, K_cat, V_the, V_cat

Only compute K, V for the NEW token each step.
Q is always fresh (only need it for current token anyway).

Why is the first token slow?

When you send a message to ChatGPT:

FIRST token (slow):
→ Process your ENTIRE prompt
→ Compute K, V for every token in prompt
→ Fill the KV cache
→ Generate first response token

SUBSEQUENT tokens (fast):
→ K, V for prompt already cached
→ Only compute for new token
→ Much less work

Time to first token' and 'tokens per second' are different metrics. Now you know why.

KV Cache grows with context:

100 tokens → small cache, fast
10,000 tokens → large cache, slower
100,000 tokens → HUGE cache, memory problems

Each layer stores K and V for all tokens.
GPT-4 has ~120 layers.
Context of 128K tokens = massive memory.

This is why long context is hard.

One Head Isn't Enough

Single attention head:
- One set of Q, K, V weights
- One "perspective" on relationships

Problem: Language has MANY types of relationships
- Syntactic: subject → verb
- Semantic: pronoun → referent
- Positional: nearby words
- Topical: related concepts

One head can't capture all of these.

Multi-Head Attention

Multi-head attention:

Head 1: Maybe learns syntax (subject-verb)
Head 2: Maybe learns coreference (it → animal)
Head 3: Maybe learns local context (nearby words)
Head 4: Maybe learns topic relationships
…

Each head has its OWN Q, K, V weights.
Each head sees different patterns.

Instead of:
  d_model = 768, one attention with 768-dim Q, K, V

We do:
  8 heads, each with 96-dim Q, K, V (768 ÷ 8 = 96)

Each head:
  Q_i = X @ W_Q_i   (produces 96-dim queries)
  K_i = X @ W_K_i   (produces 96-dim keys)
  V_i = X @ W_V_i   (produces 96-dim values)

  output_i = Attention(Q_i, K_i, V_i)

Final: Concatenate all heads → project back to 768

Why 8? And not more?

Trade-offs:

More heads = more perspectives
But each head = smaller dimension

8 heads × 96 dims = 768 total
16 heads × 48 dims = 768 total

Too many heads → each is too small to capture anything
Too few heads → not enough perspectives

Typical: 8-32 heads in practice